Module 5: Simulations and Parallel Computing

Benjamin Smith

07/21/2026

Outline

In this module, we will review

- Simulation Study
- Rationale for Simulations
- Parallel Computing in R

Simulation study

- Simulation: A numerical techniques for conducting experiments on the computer
- Monte Carlo simulation: Computer experiment involving random sampling from probability distributions

Why simulation?

To establish/validate the properties of statistical methods

- Exact analytical derivations of properties are **rarely** possible
- Large sample approximations to properties are **often possible**, but need to evaluate their relevance to (finite) sample sizes likely to be encountered in practice

Why simulation?

To establish/validate the properties of statistical methods

- Exact analytical derivations of properties are **rarely** possible
- Large sample approximations to properties are **often possible**, but need to evaluate their relevance to (finite) sample sizes likely to be encountered in practice

Moreover, analytical results may require **assumptions** (e.g., normality)

- But what happens when these assumptions are violated?
- Analytical results, even large sample ones, may not be possible

Considerations for simulation

- Is an estimator **biased** in finite samples? Is it still **consistent** under departures from assumptions? What is its **sampling variance**?
- How does it **compare** to competing estimators on the basis of bias, precision, etc.?

Considerations for simulation

- Is an estimator **biased** in finite samples? Is it still **consistent** under departures from assumptions? What is its **sampling variance**?
- How does it **compare** to competing estimators on the basis of bias, precision, etc.?
- Does a procedure for constructing a **confidence interval** for a parameter achieve the advertised **nominal level of coverage**?
- Does a **hypothesis testing** procedure attain the advertised **level** or **size**?
- If it does, what **power** is possible against different alternatives to the null hypothesis? Do different test procedures deliver different power?

Monte Carlo simulation

- Generate S independent data sets under the conditions of interest
- Compute the numerical value of the estimator/test statistic T (data) for each data set $\Rightarrow T_1, \ldots, T_S$
- If S is large enough, **summary statistics** across $T_1, \ldots, T_S$ should be good **approximations** to the true sampling properties of the estimator/test statistic under the conditions of interest

Simulations for properties of estimators

Example: Compare 3 estimators for the **mean** μ of a distribution based on i.i.d. draws $Y_1, \ldots, Y_n$

- Sample mean $T^{(1)}$
- Sample 20% trimmed mean $T^{(2)}$
- Sample median $T^{(3)}$

Simulations for properties of estimators (cont'd)

Simulation procedure: For a particular choice of μ, n, and true underlying distribution

- Generate independent draws $Y_1, \ldots, Y_n$ from the distribution
- Compute $T^{(1)}, T^{(2)}, T^{(3)}$
- Repeat S times $T_1^{(1)}, \ldots, T_S^{(1)}; \quad T_1^{(2)}, \ldots, T_S^{(2)}; \quad T_1^{(3)}, \ldots, T_S^{(3)}$
- Compute for $k = 1, 2, 3$

$$\widehat{\mu} = S^{-1} \sum_{s=1}^{S} T_s^{(k)} = \bar{T}^{(k)}, \ \widehat{\text{bias}} = \bar{T}^{(k)} - \mu$$

$$\widehat{\sigma} = \sqrt{(S-1)^{-1} \sum_{s=1}^{S} \left(T_s^{(k)} - \bar{T}^{(k)} \right)^2}$$

$$\widehat{\text{MSE}} = S^{-1} \sum_{s=1}^{S} \left(T_s^{(k)} - \mu \right)^2 \approx \widehat{\text{SD}}^2 + \widehat{\text{bias}}^2$$

Simulations for properties of estimators (cont'd)

Another important property we care about is the **relative efficiency** (RE).

- If the estimators are unbiased,

$$RE = \frac{\text{var}\left(T^{(1)}\right)}{\text{var}\left(T^{(2)}\right)}$$

- If the estimators are biased,

$$RE = \frac{\text{MSE}\left(T^{(1)}\right)}{\text{MSE}\left(T^{(2)}\right)}$$

In either case $RE < 1$ means estimator 1 is preferred (estimator 2 is inefficient relative to estimator 1 in this sense)

Set up parameters

```r
set.seed(123)
# number of simulations
S <- 1e5
# sample size
n <- 1000
# mu and sigma
mu <- 1
sigma <- sqrt(5 / 3)
# function
trimmean <- function(Y) mean(Y, 0.2)
```

Run Simulation (for loop)

```
start_time <- Sys.time()
t1 <- t2 <- t3 <- c()
for (s in 1:S) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- c(t1, mean(dat))
  # calculate T2
  t2 <- c(t2, trimmean(dat))
  # calculate T3
  t3 <- c(t3, median(dat))
}
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 47.62163 secs
```

Bias?

```
mean(t1 - 1)
```

```
## [1] 0.0002025304
```

```
mean(t2 - 1)
```

```
## [1] 0.0001590339
```

```
mean(t3 - 1)
```

```
## [1] 6.529179e-05
```

- All estimators are shown minimal bias, why?

Sample Variance?

```
var(t1)
```

```
## [1] 0.001661072
```

```
var(t2)
```

```
## [1] 0.001902612
```

```
var(t3)
```

```
## [1] 0.00261475
```

Relative Efficiency?

```
cat("T1 vs T2", (mean(t2 - 1)^2 + var(t2)) /
  (mean(t1 - 1)^2 + var(t1)), "\n")
```

```
## T1 vs T2 1.145398
```

```
cat("T1 vs T3", (mean(t3 - 1)^2 + var(t3)) /
  (mean(t1 - 1)^2 + var(t1)), "\n")
```

```
## T1 vs T3 1.574098
```

```
cat("T2 vs T3", (mean(t3 - 1)^2 + var(t3)) /
  (mean(t2 - 1)^2 + var(t2)), "\n")
```

```
## T2 vs T3 1.374279
```

Run Simulation (lapply)

```
start_time <- Sys.time()
t <- lapply(1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 12.91134 secs
```

```
# convert t to a dataframe with column t1, t2, t3
t_final <- do.call(rbind, t)
```

Run Simulation (Vectorize)

```
generate.normal <- function(S, n, mu, sigma) {
  dat <- matrix(rnorm(n * S, mu, sigma), ncol = n, byrow = T)
  out <- list(dat = dat)
  return(out)
}
```

```
start_time <- Sys.time()
out <- generate.normal(S, n, mu, sigma)
out_mean <- apply(out$dat, 1, mean)
out_trimmean <- apply(out$dat, 1, trimmean)
out_median <- apply(out$dat, 1, median)
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 14.57823 secs
```

Introduction to Embarrassing Parallelism

- `for loop` execute each task sequentially
- Modern computers are built in with multiple cores that allows you do the above jobs in parallel
- Rise of high performance computing (HPC) cluster
- The improvement is not linear!

Parallel in local computer (foreach)

- most intuitive parallel algorithm, just like `for loop`
- need to set-up the local cluster

```
library(doParallel)
```

```
## Loading required package: foreach
```

```
## Loading required package: iterators
```

```
## Loading required package: parallel
```

```
detectCores()
```

```
## [1] 16
```

```
cl <- makeCluster(8)
registerDoParallel(cl)
start_time <- Sys.time()
t <- foreach(s = 1:S, .combine = "rbind") %dopar% {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)

  c(t1, t2, t3)
}
```

Parallel in local computer (mclapply)

- The `mclapply()` function essentially parallelizes calls to `lapply()`
- Note: This does not work on Windows

```
library(parallel)

start_time <- Sys.time()
t <- mclapply(1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
}, mc.cores = 4)
end_time <- Sys.time()
end_time - start_time

# convert t to a dataframe with column t1, t2, t3
t_final <- do.call(rbind, t)
```

Parallel in local computer (parLapply)

```
cl <- makeCluster(8)
registerDoParallel(cl)
start_time <- Sys.time()
t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})
```

```
## Error in checkForRemoteErrors(val): 8 nodes produced errors; first error: object 'n' not found
```

```
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 0.0117929 secs
```

```
# convert t to a data frame with column t1, t2, t3
t_final <- do.call(rbind, t)
```

```
## Error in do.call(rbind, t): second argument must be a list
```

parLapply continued

- need to export the environment

```
clusterExport(cl, varlist = c("n", "mu", "sigma", "trimmean"))
start_time <- Sys.time()
t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 2.094134 secs
```

```
# convert t to a data frame with column t1, t2, t3
t_final <- do.call(rbind, t)
```

Error Handling (foreach)

```
t <- foreach(
  i = 1:1e4, .combine = "rbind"
) %dopar% {
  # generate data
  A <- matrix(data = rbinom(4, 1, 0.5), nrow = 2)
  solve(A)
}
```

```
## Error in {: task 2 failed - "Lapack routine dgesv: system i
```

- The error may only occur occasionally
- You want to ignore the error and finish your job

Error Handling (foreach)

```
t <- foreach(
  i = 1:1e4,
  .errorhandling = "pass"
) %dopar% {
  # generate data
  A <- matrix(data = rbinom(4, 1, 0.5), nrow = 2)
  solve(A)
}

head(t, 2)
```

```
## [[1]]
##      [,1] [,2]
## [1,]    0    1
## [2,]    1   -1
##
## [[2]]
```

Error Handling (foreach)

```
t <- foreach(
  i = 1:1e4,
  .errorhandling = "remove"
) %dopar% {
  # generate data
  A <- matrix(data = rbinom(4, 1, 0.5), nrow = 2)
  solve(A)
}
head(t, 2)
```

```
## [[1]]
##      [,1] [,2]
## [1,]    1   -1
## [2,]    0    1
##
## [[2]]
##      [,1] [,2]
## [1,]    1   -1
## [2,]    0    1
```

Error Handling (tryCatch)

- `tryCatch` enables you to handle **errors** and **warnings**

```
t <- parLapply(cl, X = 1:1e4, fun = function(x) {
  # generate data
  tryCatch(
    {
      A <- matrix(data = rbinom(4, 1, 0.5), nrow = 2)
      solve(A)
    },
    error = function(e) {
      # code that will be executed in the event of an error
      return(NA)
    }
  )
})

head(t, 2)
```

```
## [[1]]
## [1] NA
##
## [[2]]
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

Error Handling (tryCatch)

- Often times, warning messages are not outputed in the parallel process

```
sigma <- -1
t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})
head(t, 2)
```

```
## [[1]]
## [1] 1.022114 1.027980 1.067007
##
## [[2]]
## [1] 0.9218276 0.9190672 0.9434081
```

Error Handling (tryCatch)

```
sigma <- -1
t <- parLapply(cl = cl, X = 1:S, function(s) {
  tryCatch(
    {
      # generate data
      dat <- rnorm(n, mu, sigma)
      # calculate T1
      t1 <- mean(dat)
      # calculate T2
      t2 <- trimmean(dat)
      # calculate T3
      t3 <- median(dat)
      c(t1, t2, t3)
    },
    warning = function(w) {
      # code that will be executed in the event of a warning
      return(w)
    }
  )
})
head(t, 2)
```

```
## [[1]]
## [1] 0.9607585 0.9614965 0.9746650
##
## [[2]]
## [1] 1.059974 1.076250 1.092904
```

Parallel in HPC

- Using Niagara cluster (Compute Canada) as an example, it contains 2024 nodes, each with 40 cores, for a total of 80,640 cores.
- Say if you want to request 20 cores, there are two ways to request it
  - 1 node and all 20 cores on the node
  - different nodes

One node Prallel

```r
cl <- makeCluster(20)
registerDoParallel(cl)
clusterExport(cl, varlist = c("n", "mu", "sigma", "trimmean"))
t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})

# convert t to a data frame with column t1, t2, t3
t_final <- do.call(rbind, t)

# save the results
saveRDS(t_final, "t_final.rds")
```

save the R script as `example.R`

One node Prallel

Use `module spider r` to check the requirement for loading R

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=20
#SBATCH --time=0-01:30            # time (DD-HH:MM)
module load gcc/9.3.0 r/4.0.2

Rscript example.R
```

Save it as `submit.sh`

Submit the job by `sbatch submit.sh`

Multiple Nodes

- things are much more complicated
- sometimes cannot be avoided, say if you want to request 800 cores
- need to use `OpenMPI`

Multiple Nodes

```
cl <- makeCluster(800, type = "MPI")
registerDoParallel(cl)
t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})

# convert t to a data frame with column t1, t2, t3
t_final <- do.call(rbind, t)

# save the results
saveRDS(t_final, "t_final.rds")
```

save the R script as `example.R`

Multiple Nodes

```
#!/bin/bash
#SBATCH --nodes=20
#SBATCH --ntasks-per-node=40
#SBATCH --time=0-01:30            # time (DD-HH:MM)
module load gcc/9.3.0 openmpi/4.0.3 r/4.0.2

R_PROFILE=${HOME}/R/x86_64-pc-linux-gnu-library/4.0/snow/
RMPISNOWprofile;
export R_PROFILE
mpirun -np 800 -bind-to core:overload-allowed R CMD BATCH
--no-save example.R
```

Save it as `submit.sh`

Submit the job by `sbatch submit.sh`

Passing argument

- sometimes you may want to run for a set of arguments
- e.g. n = c(100, 200, 300, 400)

```
args <- commandArgs(TRUE)
n <- args[1]

cl <- makeCluster(800, type = "MPI")
registerDoParallel(cl)
clusterExport(cl, varlist = c("n", "mu", "sigma", "trimmean"))

t <- parLapply(cl = cl, X = 1:S, function(s) {
  # generate data
  dat <- rnorm(n, mu, sigma)
  # calculate T1
  t1 <- mean(dat)
  # calculate T2
  t2 <- trimmean(dat)
  # calculate T3
  t3 <- median(dat)
  c(t1, t2, t3)
})

# convert t to a data frame with column t1, t2, t3
t_final <- do.call(rbind, t)
# save the results
saveRDS(t_final, "t_final.rds")
```

Passing argument

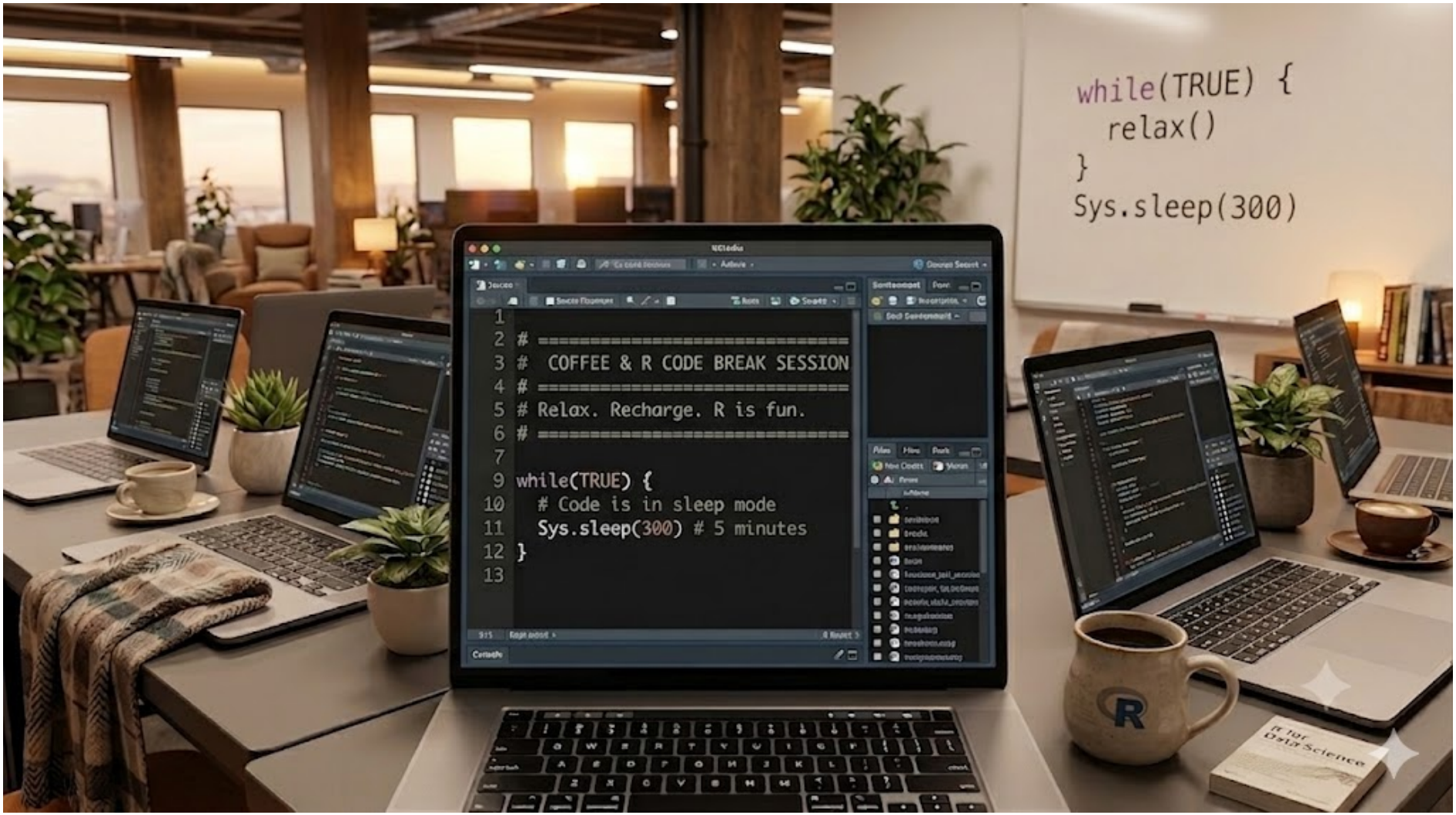
```
#!/bin/bash
#SBATCH --nodes=20
#SBATCH --ntasks-per-node=40
#SBATCH --time=0-01:30            # time (DD-HH:MM)
module load gcc/9.3.0 openmpi/4.0.3 r/4.0.2

R_PROFILE=${HOME}/R/x86_64-pc-linux-gnu-library/4.0/snow/RMPISNOWprofile; export R_PROFILE
mpirun -np 800 -bind-to core:overload-allowed R CMD BATCH --no-save "--args $n" example.R
```

Save it as `submit.sh`

Submit the job by

```
for n in 100 200 300 400
do
  sbatch --export=n=$n submit.sh
done
```

Break


while(TRUE) {
relax()
}
Sys.sleep(300)
# ==========
# COFFEE & R CODE BREAK SESSION
# ==========
# Relax. Recharge. R is fun.
# ==========
while(TRUE) {
# Code is in sleep mode
Sys.sleep(300) # 5 minutes
}

Outline

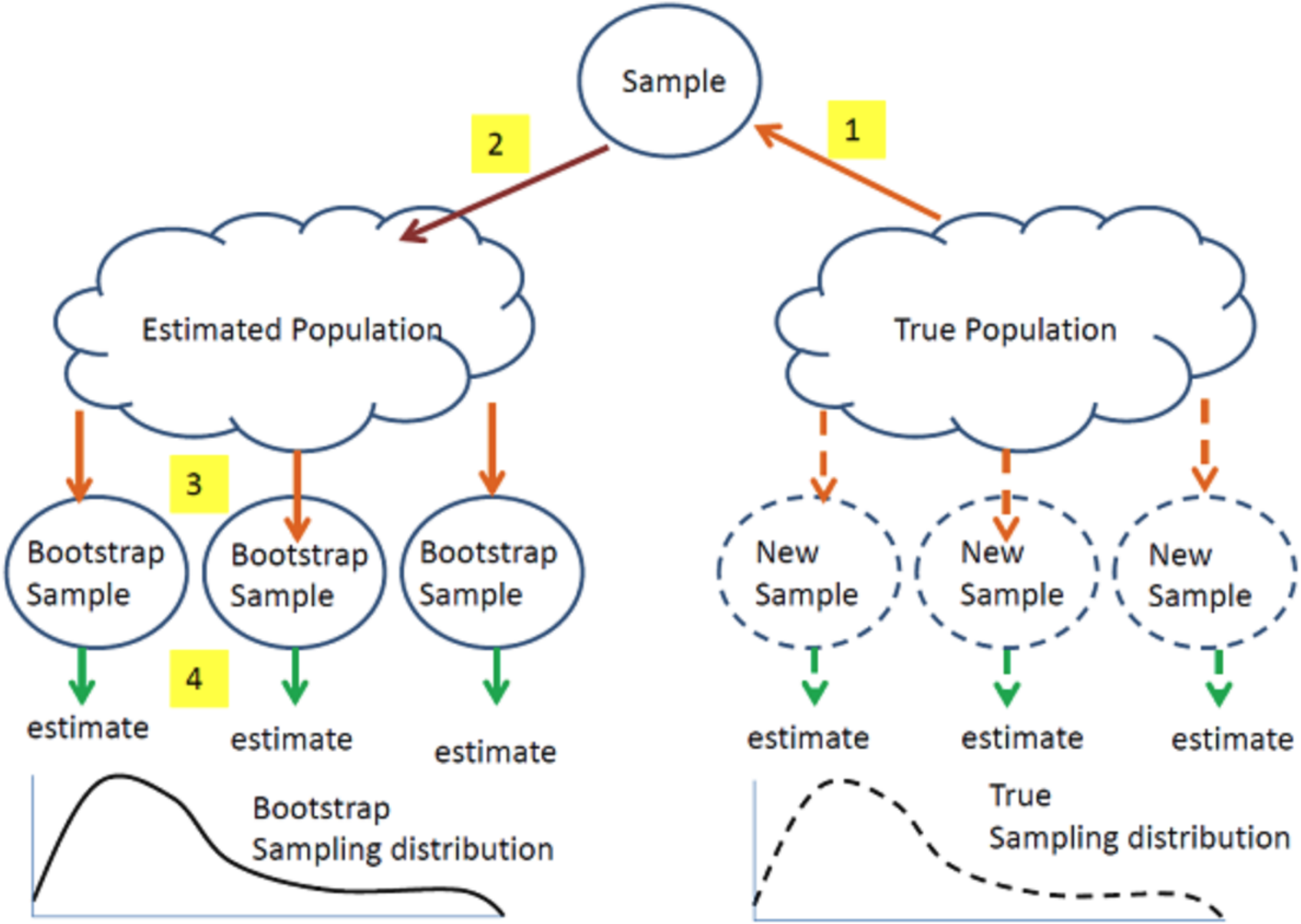
In this module, we will continue our discussion on Bootstrap.

What is bootstrap?

A widely applicable, computer intensive resampling method used to compute standard errors, confidence intervals, and significance tests.

Why bootstrap?

- The exact sampling distribution of an estimator can be **difficult** to obtain
- Asymptotic expansions are sometimes easier but expressions for standard errors based on large sample theory may not perform well in finite samples



https://online.stat.psu.edu/stat555/node/119/

Example: estimate the variance of an estimator

Now let $\text{Var}_F(T_n)$ denote the variance of T_n. The subscript F means the variance is a function of F where F is the CDF of X. If we know F, we can directly compute the variance. For example, if then $x_1, \cdots, x_w$ iid

$$T_n = \bar{X}_n, = \frac{1}{n}\sum_{i=1}^{n} x_i$$

$$\text{Var}_F(T_n) = n^{-1}\left(\int x^2 dF(x) - \left(\int x dF(x)\right)^2\right)$$

which is a function of F.

Example: estimate the variance of an estimator

When F is unknown, one can use an estimate of F, e.g., the empirical CDF $\hat{F}_n$, i.e.,

$$\hat{F}_n(x) = \frac{\sum_{i=1}^{n} \mathbf{1}(X_i \leq x)}{n}.$$

We then use a plug-in estimator for $\mathrm{Var}_{\hat{F}_n}(T_n)$:

$$\mathrm{Var}_{\hat{F}_n}(T_n) = n^{-1}\left(\int x^2 d\hat{F}_n(x) - \left(\int x d\hat{F}_n(x)\right)^2\right).$$

Example: estimate the variance of an estimator

However, the plug-in estimator above can be hard to compute, we can then approximate it with a simulation estimate, denoted by V_{boot} .

The following algorithm illustrate how one can do this through bootstrap.

Algorithm 1: Bootstrap variance estimation algorithm

Input data $(X_1, \ldots, X_n)$; Number of iteration B;
for $i \leftarrow 0$ to B do
    Draw $X^*_{1,i}, \ldots, X^*_{n,i} \sim \hat{F}_n$;
    Compute $T^*_{n,i} = g\left(X^*_{1,i}, \ldots, X^*_{n,i}\right)$;
end
$V_{\text{boot}} = \frac{1}{B}\sum_{i=1}^{B}\left(T^*_{n,i} - \frac{1}{B}\sum_{j=1}^{B} T^*_{n,j}\right)^2$.

By law of large numbers, we have $V_{\text{boot}} \xrightarrow{\text{a.s.}} \text{Var}_{\hat{F}_n}(T_n)$ as $B \to \infty$.

The bootstrap principle

Suppose $X = \{X_1, \ldots, X_n\}$ is a sample used to estimate some parameter $\theta = T(P)$ of the underlying distribution P. To make inference on θ, we are interested in the properties of our estimator $\hat{\theta} = S(X)$ for θ.

The bootstrap principle

Suppose $X = \{X_1, \ldots, X_n\}$ is a sample used to estimate some parameter $\theta = T(P)$ of the underlying distribution P. To make inference on θ, we are interested in the properties of our estimator $\hat{\theta} = S(X)$ for θ.

- If we knew P,
  - we could obtain $\left\{X^b \mid b = 1, \ldots B\right\}$ from P and use Monte-Carlo to estimate the sampling distribution of $\hat{\theta}$

The bootstrap principle

Suppose $X = \{X_1, \ldots, X_n\}$ is a sample used to estimate some parameter $\theta = T(P)$ of the underlying distribution P. To make inference on θ, we are interested in the properties of our estimator $\hat{\theta} = S(X)$ for θ.

- If we knew P,
  - we could obtain $\left\{X^b \mid b = 1, \ldots B\right\}$ from P and use Monte-Carlo to estimate the sampling distribution of $\hat{\theta}$
- However, we don't,
  - we do the next best thing and resample from original sample, i.e. the empirical distribution, $\hat{P}$
  - we expect the empirical distribution to estimate the underlying distribution well by the *Glivenko-Cantelli* Theorem

Why bootstrap works

Definition 2.1. Let F, G be two CDF's on a sample space X. Let $\rho(F, G)$ be a metric on the space of CDF's on X. We say G_n^* is weakly ρ-consistent if

$$\rho\left(G_n^*, G_n\right) \xrightarrow{P} 0$$

as $n \to \infty$. Similarly, G_n^* is strongly ρ-consistent if

$$\rho\left(G_n^*, G_n\right) \xrightarrow{\text{a.s.}} 0.$$

Why bootstrap works

Now the measure of closeness between two CDF's depends on the metric ρ. The following two metrics are commonly used for the CDF's:

(i) Kolmogorov metric:

$$K(F, G) = \sup_{x \in R} |F(x) - G(x)|.$$

(ii) Mallows-Wasserstein metric:

$$l_2(F, G) = \inf_{T_{F,G}} \left(E|Y - X|^2 \right)^{1/2},$$

where $T_{F,G}$ is the collection of all possible joint distribution of the pair (X, Y) whose marginal distributions are F, G. The following theorem illustrates a special case of the bootstrap theory.

Why bootstrap works

Theorem. Suppose $X_1, \ldots, X_n \overset{\text{i.i.d.}}{\sim} F$ and $E\left(X_i^2\right) < \infty$. Let

$$T_n = g\left(X_1, \ldots, X_n\right) = \sqrt{n}(\bar{X} - \mu).$$

For the bootstrap version $T_n^* = \sqrt{n}(\bar{X}_n^* - \bar{X}_n)$:

$$K(G_n^*, G_n) \xrightarrow{a.s.} 0 \quad \text{and} \quad l_2(G_n^*, G_n) \xrightarrow{a.s.} 0$$

where:

$$G_n(t) = P(T_n \leq t) \quad \text{(True CDF)}$$
$$G_n^*(t) = P(T_n^* \leq t \mid X_1, \ldots, X_n) \quad \text{(Bootstrap CDF)}$$

Practical Implementation

Monte Carlo Approximation

For finite samples, estimate G_n^* via:

$$\hat{G}_n^*(t) = \frac{1}{B} \sum_{b=1}^{B} \mathbb{I}(T_{n,b}^* \leq t)$$

where:

- B = number of bootstrap samples
- $T_{n,b}^* = \sqrt{n}(\bar{X}_{n,b}^* - \bar{X}_n)$

Convergence

$$\begin{array}{ll} B \to \infty & \hat{G}_n^* \to G_n^* \\ n \to \infty & G_n^* \to G_n \end{array}$$

Bootstrap confidence intervals

For a statistic T_n with bootstrap replicates $\{T^*_{n,b}\}_{b=1}^B$:

- **Normal Approximation CI** (requires normality):

$$T_n \pm z_{\alpha/2}\widehat{se}_{boot}$$

where $\widehat{se}_{boot} = \sqrt{\frac{1}{B-1}\sum_{b=1}^B (T^*_{n,b} - \bar{T}^*)^2}$

- **Pivotal (Basic) Bootstrap CI**:

$$\left(2T_n - T^*_{((1-\alpha/2)B)},\ 2T_n - T^*_{((\alpha/2)B)}\right)$$

- **Percentile Bootstrap CI**:

$$\left(T^*_{((\alpha/2)B)},\ T^*_{((1-\alpha/2)B)}\right)$$

Bootstrap confidence intervals

Important Notes

- Normal CI assumes asymptotic normality of T_n
- Pivotal CI inverts the bootstrap distribution
- Percentile CI is simple but may be biased

Forms of bootstrap

Based on how the population is estimated,

1. Nonparametric bootstrap
2. Parametric bootstrap
3. Empirical Bootstrap (Paired Bootstrap)
4. Residual Bootstrap
5. Wild Bootstrap

Nonparametric bootstrap (resampling)

Core Idea

Resample *with replacement* from original data to approximate sampling distribution of statistic $\hat{\theta} = S(\mathbf{D})$

Nonparametric bootstrap (resampling)

Core Idea

Resample *with replacement* from original data to approximate sampling distribution of statistic $\hat{\theta} = S(\mathbf{D})$

1. **Resampling**: Generate B bootstrap datasets $\mathbf{D}^*(1), \ldots, \mathbf{D}^*(B)$ where each $\mathbf{D}^*(b)$ contains n observations drawn *with replacement* from original data $\mathbf{D} = (X_1, \ldots, X_n)$

Nonparametric bootstrap (resampling)

Core Idea

Resample *with replacement* from original data to approximate sampling distribution of statistic $\hat{\theta} = S(\mathbf{D})$

1. **Resampling**: Generate B bootstrap datasets $\mathbf{D}^*(1), \ldots, \mathbf{D}^*(B)$ where each $\mathbf{D}^*(b)$ contains n observations drawn *with replacement* from original data $\mathbf{D} = (X_1, \ldots, X_n)$
2. **Replication**: Compute bootstrap statistics $\hat{\theta}^*(b) = S(\mathbf{D}^*(b))$ for $b = 1, \ldots, B$

Nonparametric bootstrap (resampling)

Core Idea

Resample *with replacement* from original data to approximate sampling distribution of statistic $\hat{\theta} = S(\mathbf{D})$

1. **Resampling**: Generate B bootstrap datasets $\mathbf{D}^*(1), \ldots, \mathbf{D}^*(B)$ where each $\mathbf{D}^*(b)$ contains n observations drawn *with replacement* from original data $\mathbf{D} = (X_1, \ldots, X_n)$
2. **Replication**: Compute bootstrap statistics $\hat{\theta}^*(b) = S(\mathbf{D}^*(b))$ for $b = 1, \ldots, B$
3. **Standard Error Estimation**:

$$\widehat{se}_{boot} = \sqrt{\frac{1}{B-1} \sum_{b=1}^{B} \left(\hat{\theta}^*(b) - \bar{\theta}^* \right)^2}$$

where $\bar{\theta}^* = \frac{1}{B} \sum_{b=1}^{B} \hat{\theta}^*(b)$

Parametric bootstrap

- Assumes the data comes from a known distribution with unknown parameters
- First estimate the parameters from the data and then use the estimated distribution to simulate the samples

Bootstrap for Regression

Bootstrap can also be used to conduct statistical inference for regression model. Let $(X_1, Y_1), \ldots, (X_n, Y_n)$ be the observed data and

$$E(Y_i \mid X_i = x) = \beta_0 + \beta_1 x,$$

i.e.,

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i.$$

There are several variants to the bootstrap under regression setting.

Empirical Bootstrap (Paired Bootstrap)

We generate a new sets of i.i.d. observations $(X_1^*, Y_1^*), \ldots, (X_n^*, Y_n^*)$ such that for each I,

$$P\left(X_I^* = X_i, Y_I^* = Y_i\right) = \frac{1}{n}$$

for all i. Similar to bootstrap estimation of variance, we repeat this procedure B times and fit a linear regression model for each of the generated paired data:

$$\left(X_1^{*(1)}, Y_1^{*(1)}\right), \ldots, \left(X_n^{*(1)}, Y_n^{*(1)}\right) \xrightarrow{\text{fit linear regression}} \hat{\beta}_0^{*(1)}, \hat{\beta}_1^{*(1)}$$

$$\ldots$$

$$\left(X_1^{*(B)}, Y_1^{*(B)}\right), \ldots, \left(X_n^{*(B)}, Y_n^{*(B)}\right) \xrightarrow{\text{fit linear regression}} \hat{\beta}_0^{*(B)}, \hat{\beta}_1^{*(B)}.$$

Empirical Bootstrap (Paired Bootstrap)

We then estimate the bootstrap variance

$$\hat{\text{Var}}\left(\hat{\beta}_0\right) = \frac{1}{B}\sum_{l=1}^{B}\left(\hat{\beta}_0^{*(l)} - \bar{\beta}_0^*\right)^2, \bar{\beta}_0^* = \frac{1}{B}\sum_{i=1}^{B}\hat{\beta}_0^{*(l)},$$

and

$$\hat{\text{Var}}\left(\hat{\beta}_1\right) = \frac{1}{B}\sum_{l=1}^{B}\left(\hat{\beta}_1^{*(l)} - \bar{\beta}_1^*\right)^2, \bar{\beta}_1^* = \frac{1}{B}\sum_{l=1}^{B}\hat{\beta}_1^{*(l)}.$$

We can also obtain the bootstrap CI as

$$\hat{\beta}_0 \pm z_{1-\alpha/2}\sqrt{\hat{\text{Var}}\left(\hat{\beta}_0\right)}$$

and

$$\hat{\beta}_1 \pm z_{1-\alpha/2}\sqrt{\hat{\text{Var}}\left(\hat{\beta}_1\right)}$$

Empirical Bootstrap (Paired Bootstrap)

Although empirical bootstrap works well in practice, it may lead to a bad results, especially in the presence of influential observations (some X_i 's are very far away from the others).

This will also strongly affect the empirical bootstrap in the sense that if an empirical bootstrap does not select these influential points, the regression coefficients can be very different.

Residual Bootstrap

To solve the problem of empirical bootstrap illustrated previously, we may use the residual bootstrap. We first fit the original data to obtain the OLS estimate $\hat{\beta}_0, \hat{\beta}_1$. Define

$$e_i = Y_i - \hat{Y}_i = Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i.$$

e_i here are the fitted residuals and can be regarded as a good approximation of ϵ_i. The residual bootstrap generate i.i.d. $\hat{\epsilon}_1^*, \ldots, \hat{\epsilon}_n^*$ such that for each $\hat{\epsilon}_l^*$,

$$P\left(\hat{\epsilon}_l^* = e_i\right) = \frac{1}{n}.$$

Then we generate a new bootstrap sample

$$\left(X_1^*, Y_1^*\right), \ldots, \left(X_n^*, Y_n^*\right)$$

via

$$X_i^* = X_i, Y_i^* = \hat{\beta}_0 + \hat{\beta}_1 X_i + \hat{\epsilon}_i^*.$$

Wild Bootstrap

Another issue usually occur in linear regression is the so called phenomenon of heteroskedasticity, i.e., $\text{Var}(\epsilon_i \mid X_i)$ depends on the value of X_i.

Now residual bootstrap cannot deal with this issue. In particular, the residual bootstrap will be unstable because the residual bootstrap will swap all the residuals regardless of the value of the covariate. The solution to this is through the wild bootstrap.

The wild bootstrap first generate i.i.d. random variables $V_1, \ldots, V_n \sim \mathcal{N}(0, 1)$ and then generate the bootstrap sample

$$(X_1^*, Y_1^*), \ldots, (X_n^*, Y_n^*)$$

by

$$Y_i^* = \hat{\beta}_0 + \hat{\beta}_1 X_i + V_i e_i, X_i^* = X_i.$$

Bootstrap hypothesis testing

```
boot_dat <- function(x, n, mu, std) {
  newx <- sample(x, n, replace = T)
  return((mean(newx) - mu)/(std/sqrt(n)))
}


pvalues_1 <- rep(NA, nsim)
pvalues_2 <- rep(NA, nsim)

for (i in 1:nsim){
  x <- rnorm(n, mean = mu, sd = std)
  stat <- abs((mean(x) - mu)/(std/sqrt(n)))
  pvalues_1[i] <- 2*(1 - pnorm(stat))


  newx <- x - mean(x) + mu
  boot_scores <- sapply(1:nboot,
                        function(index) boot_dat(newx, n, mu, std))
  boot_scores <- as.vector(boot_scores)
  # hist(boot_scores)
  # mean(boot_scores)
  pvalues_2[i] <- length(which(boot_scores > stat))/length(boot_scores) +
    length(which(boot_scores < (-stat)))/length(boot_scores)
}
```

Failures of Bootstrap

Let $X_1, \ldots, X_n \overset{\text{i.i.d.}}{\sim} \text{unif}\,(0,1)$ and $M_n = \min(X_1, \ldots, X_n)$ be the minimum of the sample. One can show that

$$nM_n \xrightarrow{\mathcal{D}} \exp(1)$$

However, let $M_n^* = \min(X_1^*, \ldots, X_n^*)$ be the minimum of a bootstrap sample, then

$$P(\text{ none of } X_1^*, \ldots, X_n^* \text{ select } M_n) = \left(1 - \frac{1}{n}\right)^n \approx e^{-1},$$

which implies that

$$P(M_n^* = M_n) \approx 1 - e^{-1}.$$

Therefore, M_n^* has a huge probability mass at M_n. This means the distribution of M_n^* is very different from the distribution of M_n. The detailed proof is left as homework.

StatQuest videos

Check out these videos made by Josh Starmer with vivid illustration for the boostrap!

- Bootstrapping Main Ideas [link]
- Using Bootstrapping to Calculate p-values [link]

Resources

This tutorial is based on

- PennState STAT555 Statistical Analysis of Genomics Data [links].
- Harvard's Biostatistics Preparatory Course Methods [links].